

DATA ANALYSIS COMPARISON

SQL

Select * from dataset_1 d

select weather, temperature from dataset_1 d

A-Z weather

123 temperature

1	Sunny	55
2	Sunny	80
3	Sunny	80
4	Sunny	80
5	Sunny	80
6	Sunny	80
7	Sunny	55
8	Sunny	80
9	Sunny	80
10	Sunny	80
11	Sunny	80
12	Sunny	55

PYTHON

df

	destination	passanger	weather	temperature	time	coupon	expiration	gender	age	maritalStatus	...	CarryAway	Restaurant
0	No Urgent Place	Alone	Sunny	55	2PM	Restaurant(<20)	1d	Female	21	Unmarried partner	...	NaN	
1	No Urgent Place	Friend(s)	Sunny	80	10AM	Coffee House	2h	Female	21	Unmarried partner	...	NaN	
2	No Urgent Place	Friend(s)	Sunny	80	10AM	Carry out & Take away	2h	Female	21	Unmarried partner	...	NaN	
3	No Urgent Place	Friend(s)	Sunny	80	2PM	Coffee House	2h	Female	21	Unmarried partner	...	NaN	
4	No Urgent Place	Friend(s)	Sunny	80	2PM	Coffee House	1d	Female	21	Unmarried partner	...	NaN	
...	...	...	...	...	...	...	...	...	...	...	...	...	...
12679	Home	Partner	Rainy	55	6PM	Carry out & Take away	1d	Male	26	Single	...	1-3	
12680	Work	Alone	Rainy	55	7AM	Carry out & Take away	1d	Male	26	Single	...	1-3	
12681	Work	Alone	Snowy	30	7AM	Coffee House	1d	Male	26	Single	...	1-3	
12682	Work	Alone	Snowy	30	7AM	Bar	1d	Male	26	Single	...	1-3	
12683	Work	Alone	Sunny	80	7AM	Restaurant(20-50)	2h	Male	26	Single	...	1-3	

12684 rows x 27 columns

1. select distinct passanger from dataset_1 d ;

A-Z passange

Alone

Friend(s)

Kid(s)

Partner

df['passanger'].unique()

[8]:

array(['Alone', 'Friend(s)', 'Kid(s)', 'Part

```
*select weather,temperature FROM dataset_1
```

AZ weather	123 temperature
Sunny	
Sunny	
Sunny	
Sunny	
Sunny	
Sunny	
Sunny	
Sunny	
Sunny	
Sunny	
Sunny	

```
df[['weather','temperature']]
```

[6]:

	weather	temperature
0	Sunny	55
1	Sunny	80
2	Sunny	80
3	Sunny	80
4	Sunny	80
...	...	...
12679	Rainy	55
12680	Rainy	55
12681	Snowy	30
12682	Snowy	30
12683	Sunny	80

12684 rows x 2 columns

```
select * from dataset_1 limit 10
```

	AZ destination	AZ passanger	AZ weather
1	No Urgent Place	Alone	Sunny
2	No Urgent Place	Friend(s)	Sunny
3	No Urgent Place	Friend(s)	Sunny
4	No Urgent Place	Friend(s)	Sunny
5	No Urgent Place	Friend(s)	Sunny
6	No Urgent Place	Friend(s)	Sunny
7	No Urgent Place	Friend(s)	Sunny
8	No Urgent Place	Kid(s)	Sunny
9	No Urgent Place	Kid(s)	Sunny
10	No Urgent Place	Kid(s)	Sunny

```
df.head(10)
```

[6]:

	destination	passanger	weather	temperature	time	coupon	expiration
0	No Urgent Place	Alone	Sunny	55	2PM	Restaurant(<20)	1d
1	No Urgent Place	Friend(s)	Sunny	80	10AM	Coffee House	2h
2	No Urgent Place	Friend(s)	Sunny	80	10AM	Carry out & Take away	2h
3	No Urgent Place	Friend(s)	Sunny	80	2PM	Coffee House	2h
4	No Urgent Place	Friend(s)	Sunny	80	2PM	Coffee House	1d
5	No Urgent Place	Friend(s)	Sunny	80	6PM	Restaurant(<20)	2h
6	No Urgent Place	Friend(s)	Sunny	55	2PM	Carry out & Take away	1d
7	No Urgent Place	Kid(s)	Sunny	80	10AM	Restaurant(<20)	2h
8	No Urgent Place	Kid(s)	Sunny	80	10AM	Carry out & Take away	2h
9	No Urgent Place	Kid(s)	Sunny	80	10AM	Bar	1d

10 rows x 27 columns

```
select * from dataset_1 d where destination='Home'
```

AZ destination	AZ passanger	AZ weather
Home	Alone	Sunny
Home	Alone	Sunny
Home	Alone	Sunny
Home	Alone	Sunny
Home	Alone	Sunny
Home	Alone	Sunny
Home	Alone	Sunny
Home	Alone	Sunny
Home	Alone	Sunny
Home	Alone	Sunny

```
df[df['destination']!='Home']
```

[9]:

	destination	passanger	weather	temperature	time
13	Home	Alone	Sunny	55	6PM
14	Home	Alone	Sunny	55	6PM
15	Home	Alone	Sunny	80	6PM
35	Home	Alone	Sunny	55	6PM
36	Home	Alone	Sunny	55	6PM
...	...	...	...	...	...

```
select * from dataset_1 d order by coupon
```

A-Z destination	A-Z passanger	A-Z weather
No Urgent Place	Kid(s)	Sunny
Home	Alone	Sunny
Work	Alone	Sunny
No Urgent Place	Friend(s)	Sunny
Home	Alone	Sunny
Work	Alone	Sunny
No Urgent Place	Friend(s)	Sunny
Home	Alone	Sunny
Work	Alone	Sunny
No Urgent Place	Kid(s)	Sunny

```
df.sort_values('coupon')
```

[10]:

	destination	passanger	weather	temperature	time	coupon
11702	Home	Partner	Sunny	30	10PM	Bar
9930	No Urgent Place	Alone	Snowy	30	2PM	Bar
10632	Home	Alone	Rainy	55	6PM	Bar
7997	No Urgent Place	Friend(s)	Rainy	55	10PM	Bar
11166	Work	Alone	Snowy	30	7AM	Bar
...	...	...	...	...	...	...
10476	Home	Alone	Sunny	80	6PM	Restaurant(<20)
5447	Home	Alone	Sunny	80	10PM	Restaurant(<20)
10478	Home	Alone	Snowy	30	10PM	Restaurant(<20)
5440	No Urgent Place	Alone	Sunny	80	2PM	Restaurant(<20)
0	No Urgent Place	Alone	Sunny	55	2PM	Restaurant(<20)

12684 rows x 27 columns

```
select destination as Destination from dataset_1 d
```

	A-Z Destination
1	No Urgent Pla
2	No Urgent Pla
3	No Urgent Pla
4	No Urgent Pla
5	No Urgent Pla
6	No Urgent Pla
7	No Urgent Pla
8	No Urgent Pla
9	No Urgent Pla
10	No Urgent Pla
11	No Urgent Pla

```
df.rename(columns={'destination': 'Destination'}, inplace=True)
df
```

[12]:

	Destination	passanger	weather	temperature	time	coup
0	No Urgent Place	Alone	Sunny	55	2PM	Restaurant(<20)
1	No Urgent Place	Friend(s)	Sunny	80	10AM	Coffee Hol
2	No Urgent Place	Friend(s)	Sunny	80	10AM	Carry ou Take av
3	No Urgent Place	Friend(s)	Sunny	80	2PM	Coffee Hol
4	No Urgent Place	Friend(s)	Sunny	80	2PM	Coffee Hol
...	...	...	...	...	...	...
12679	Home	Partner	Rainy	55	6PM	Carry ou Take av
12680	Work	Alone	Rainy	55	7AM	Carry ou Take av
12681	Work	Alone	Snowy	30	7AM	Coffee Hol
12682	Work	Alone	Snowy	30	7AM	Coffee Hol
12683	Work	Alone	Sunny	80	7AM	Restaurant(<20)

```
select occupation from dataset_1 d group by occupation
```

A-Z occupation

Architecture & Engineering

Arts Design Entertainment Sports & Media

Building & Grounds Cleaning & Maintenance

Business & Financial

Community & Social Services

Computer & Mathematical

Construction & Extraction

Education&Training&Library

Farming Fishing & Forestry

Food Preparation & Serving Related

```
df.groupby('occupation').size().to_frame('Count').reset_index()
```

[13]:

	occupation	Count
0	Architecture & Engineering	175
1	Arts Design Entertainment Sports & Media	629
2	Building & Grounds Cleaning & Maintenance	44
3	Business & Financial	544
4	Community & Social Services	241
5	Computer & Mathematical	1408
6	Construction & Extraction	154
7	Education&Training&Library	943
8	Farming Fishing & Forestry	43
9	Food Preparation & Serving Related	298
10	Healthcare Practitioners & Technical	244
11	Healthcare Support	242
12	Installation Maintenance & Repair	133
13	Legal	219

```
SELECT weather ,AVG(temperature) as avg_temp FROM dataset_1 GROUP BY weather
```

A-Z weather

123 avg_

Rainy

Snowy

Sunny

68.9

```
df.groupby('weather')['temperature'].mean().to_frame('avg_temp').reset_index()
```

	weather	avg_temp
0	Rainy	55.000000
1	Snowy	30.000000
2	Sunny	68.946271

```
SELECT weather ,COUNT( temperature) AS count_temp FROM dataset_1 GROUP BY weather
```

	weather	count_temp
0	Rainy	1210
1	Snowy	1405
2	Sunny	10069

```
df.groupby('weather')['temperature'].size().to_frame('Count_temp').reset_index()
```

[12]:

	weather	Count_temp
0	Rainy	1210
1	Snowy	1405
2	Sunny	10069

```
SELECT weather ,COUNT(DISTINCT temperature) AS count_distinct_temp FROM dataset_1 GROUP BY weather
```

	weather	count_distinct_temp
0	Rainy	1
1	Snowy	1
2	Sunny	3

```
df.groupby('weather')['temperature'].nunique().to_frame('count_distinct_temp').reset_index()
```

	weather	count_distinct_temp
0	Rainy	1
1	Snowy	1
2	Sunny	3

```
SELECT weather ,SUM(temperature) AS sum_temp FROM dataset_1 GROUP BY weather
```

	weather	sum_temp
0	Rainy	66550
1	Snowy	42150
2	Sunny	694220

```
df.groupby('weather')['temperature'].sum().to_frame('sum_temp').reset_index()
```

	weather	sum_temp
0	Rainy	66550
1	Snowy	42150
2	Sunny	694220

`SELECT weather ,MIN(temperature) AS min_temp FROM dataset_1 GROUP BY weather`

SQL interface showing the result of the query. The result is a table with two columns: weather and min_temp. The data is as follows:

weather	min_temp
Rainy	55
Snowy	30
Sunny	30

```
df.groupby('weather')['temperature'].min().to_frame('min_temp').reset_index()
```

	weather	min_temp
0	Rainy	55
1	Snowy	30
2	Sunny	30

`SELECT weather ,MAX(temperature) AS max_temp FROM dataset_1 GROUP BY weather`

SQL interface showing the result of the query. The result is a table with two columns: weather and max_temp. The data is as follows:

weather	max_temp
Rainy	55
Snowy	30
Sunny	80

```
df.groupby('weather')['temperature'].max().to_frame('max_temp').reset_index()
```

	weather	max_temp
0	Rainy	55
1	Snowy	30
2	Sunny	80

`SELECT occupation FROM dataset_1 GROUP BY occupation HAVING occupations='Student'`

SQL interface showing the result of the query. The result is a table with two columns: occupation and occupations. The data is as follows:

occupation	occupations
Student	Student

```
df.groupby('occupation').filter(lambda x: x['occupation'].iloc[0] == 'Student').groupby('occupation').size()
```

occupation	
Student	1584

dtype: int64

`SELECT DISTINCT destination FROM(SELECT * FROM dataset_1 UNION SELECT * FROM table_to_union)`

SQL interface showing the result of the query. The result is a table with two columns: destination and Value. The data is as follows:

destination	Value
Home	Home
No Urgent Place	
UNION	
Work	

```
pd.concat([df, df1])['destination'].drop_duplicates()
```